

PLAN DE EJECUCIÓN ACTUALIZADO

Conectividad en Juntas de Saneamiento

IMPLEMENTACIÓN DE BÚSQUEDA PRIMERO EN ANCHURA (BFS) EN PYTHON

Dato	Información verificada
Asignatura	Inteligencia Artificial
Especificación	Algoritmos de búsqueda no informada en Python
Problema	Propagación lógica de una alerta de rotura de cañería
Equipo	Mateo y Santiago
Estado actual	Implementación mínima, documentada y con 13 pruebas

Objetivo

Modelar una red hipotética de puntos de agua potable como un grafo no dirigido y determinar con BFS el orden de propagación de una alerta, completando cada nivel antes de avanzar al siguiente.

Resultado implementado

La aplicación usa sólo la **biblioteca estándar de Python**: carga el JSON, valida la red, ejecuta BFS y ofrece cinco escenarios sin modificar los datos originales.

Alcance: simulación académica. Una toma rota representa el origen de una alerta controlada; no existe detección automática de daños reales.

1. Problema convertido en búsqueda

La alerta comienza en **S1 - Tanque**. La prioridad no depende de kilómetros, presión ni costo: depende del menor número de enlaces lógicos desde el origen.

Pregunta de búsqueda

¿Qué puntos reciben primero la alerta y cuál es la ruta con menos conexiones hasta un destino?

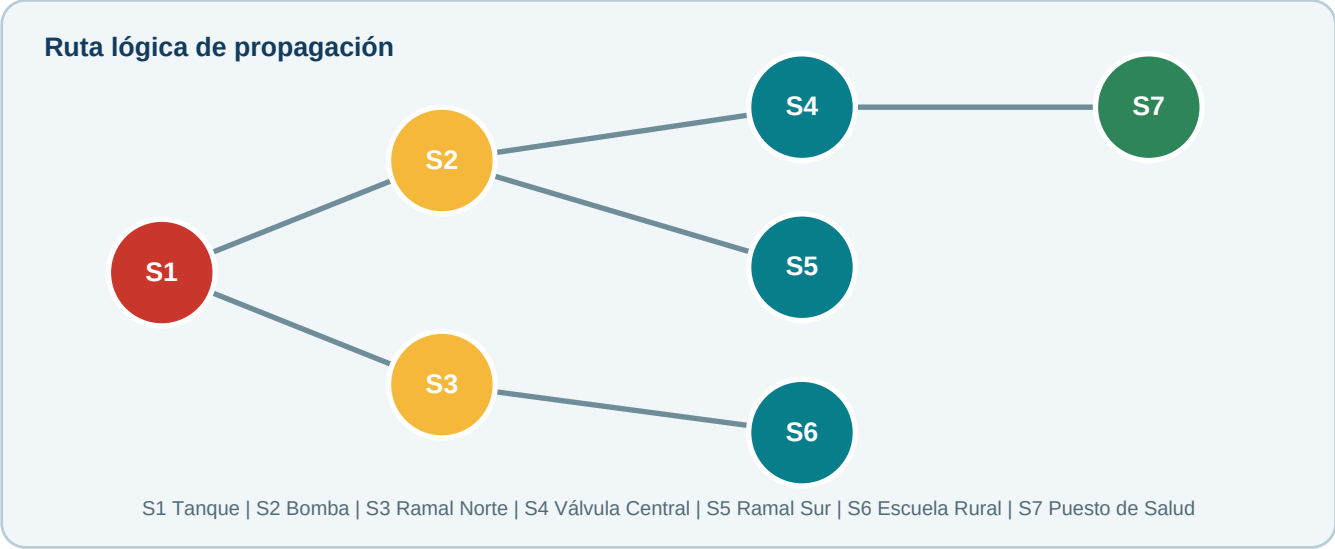
Elemento del caso	Modelo	Decisión implementada
Punto de control	Nodo	Identificadores S1 a S7
Conexión	Arista no dirigida	Se recorre en ambos sentidos
Detección	Estado inicial	Origen predeterminado S1
Propagación	Exploración	BFS por niveles
Cercanía lógica	Profundidad	Cantidad de enlaces
Evitar repeticiones	Visitados	Cada nodo se descubre una vez

Qué resuelve el repositorio

Necesidad	Respuesta actual
Orden y niveles	BFSResult guarda visit_order y levels.
Ruta hasta S7	predecessors permite reconstruir S1 -> S2 -> S4 -> S7.
Puntos aislados	unreachable informa los no alcanzables.
Datos incorrectos	network.py rechaza JSON, referencias y orden de vecinos inválidos.
Variantes didácticas	scenarios.py crea copias en memoria para S8, S9 y conexiones interrumpidas.
Errores de uso	La CLI muestra un mensaje claro y termina con código 2.

2. Red hipotética S1-S7

Los nombres representan puntos de servicio, no ubicaciones reales. La topología está versionada en `data/red_sensores.json`.



Lectura por niveles

Nivel	Puntos	Lectura
0	S1 - Tanque	Origen de la alerta
1	S2 - Bomba; S3 - Ramal Norte	Primer grupo descubierto
2	S4 - Válvula Central; S5 - Ramal Sur; S6 - Escuela Rural	Se procesa después del nivel 1
3	S7 - Puesto de Salud	Último nivel del ejemplo

Orden verificado

S1 -> S2 -> S3 -> S4 -> S5 -> S6 -> S7. El campo `neighbor_order` del JSON fija un orden reproducible dentro de cada nivel; las distancias BFS no cambian.

3. Solución algorítmica: BFS

BFS es una búsqueda no informada: no usa una heurística. La cola FIFO mantiene el orden de descubrimiento.

Paso	Acción	Resultado
1	Validar el origen	Si no existe, error controlado
2	Encolar y marcar el origen	Nivel del origen = 0
3	Extraer el primer nodo	Se agrega al orden de visita
4	Descubrir vecinos no visitados	Se asignan nivel y predecesor
5	Repetir hasta vaciar la cola	Quedan orden, niveles y rutas
6	Comparar nodos con visitados	Se obtienen los no alcanzables

Pseudocódigo

```
BFS(red, origen):
    validar origen
    cola <- [origen]; visitados <- {origen}
    nivel[origen] <- 0; predecesor[origen] <- ninguno
    mientras la cola no esté vacía:
        actual <- retirar primero
        para cada vecino de actual:
            si no está visitado: marcar, asignar nivel y predecesor, encolar
    devolver orden, niveles, predecesores y no alcanzables
```

Por qué encuentra una ruta mínima

Cuando BFS descubre un nodo del nivel k, todos los niveles anteriores ya fueron procesados. Por eso la primera ruta encontrada usa la menor cantidad de enlaces en un grafo no ponderado.

4. Implementación mínima en Python

Cada archivo tiene una tarea: cargar la red, ejecutar BFS, crear escenarios o mostrar la consola. Este extracto coincide con `src/junta_saneamiento/bfs.py`.

```
def breadth_first_search(network: Network, origin: str) -> BFSResult:
    queue = deque([origin])
    visited = {origin}
    levels = {origin: 0}
    predecessors = {origin: None}
    order = []

    while queue:
        current = queue.popleft()
        order.append(current)
        for neighbor in network.neighbors[current]:
            if neighbor not in visited:
                visited.add(neighbor)
                levels[neighbor] = levels[current] + 1
                predecessors[neighbor] = current
                queue.append(neighbor)

    unreachable = [
        node for node in network.nodes if node not in visited
    ]
    return BFSResult(order, levels, predecessors, unreachable)
```

Decisiones de eficiencia

Decisión	Beneficio
Lista de adyacencia	Memoria $O(V + E)$ para una red con pocas conexiones por nodo
<code>deque.popleft()</code>	Extracción eficiente del primer elemento
set de visitados	Consulta promedio $O(1)$ y prevención de ciclos
Biblioteca estándar	<code>argparse</code> , <code>collections</code> , <code>dataclasses</code> , <code>json</code> y <code>pathlib</code>
Escenarios separados	Permiten experimentar sin alterar el JSON original

5. Escenarios y pruebas verificadas

Cada función parte de S1-S7, crea una variante en memoria y reutiliza exactamente el mismo BFS.

Escenario	Cambio temporal	Resultado esperado
rotura-tanque	Sin cambios; origen S1	Ruta S1-S2-S4-S7
rotura-toma-s8	Añade S8 conectado a S5	Ruta S8-S5-S2-S4-S7
toma-aislada	Añade S9 sin conexiones	S9 no alcanzable
conexion-interrumpida	Retira S2-S4 en ambos sentidos	S4 y S7 no alcanzables
origen-inexistente	Intenta iniciar en S99	ValueError y salida 2

Cobertura automática

Grupo	Métodos	Qué confirma
BFS	3	Orden, ruta, ciclos, aislados y errores de origen
Carga JSON	3	Topología válida y configuraciones incorrectas
Escenarios y CLI	7	Cinco variantes, selección y opciones incompatibles
Total	13	Todos aprobados

Validación ejecutada: `python -m unittest discover -s tests -v` - resultado: **Ran 13 tests / OK.**

6. Ejecución desde consola y Python

La aplicación requiere Python 3.11 o superior. No instala ni importa paquetes externos en tiempo de ejecución.

Desde PowerShell

```
python -m pip install -e .
python -m junta_saneamiento
python -m junta_saneamiento --escenario rotura-toma-s8
python -m junta_saneamiento --escenario toma-aislada
python -m unittest discover -s tests -v
```

Desde Python

```
from junta_saneamiento import simular_rotura_en_toma_s8

escenario = simular_rotura_en_toma_s8()
print(escenario.resultado.visit_order)
print(escenario.ruta)
# ['S8', 'S5', ...]
# ['S8', 'S5', 'S2', 'S4', 'S7']
```

Reglas de uso

Regla	Motivo
No combinar --escenario con --red, --origen o --destino	Evita mezclar una variante predefinida con parámetros manuales
La función de S99 genera ValueError	Conserva el error didáctico del BFS
La red base sigue limitada a S1-S7	Las variantes se crean sólo durante la ejecución

7. Eficiencia, eficacia y calidad

Dimensión	Indicador	Resultado
Tiempo	Complejidad de BFS	$O(V + E)$
Memoria auxiliar	Cola, visitados y diccionarios	$O(V)$
Eficacia funcional	Pruebas automáticas	13 de 13
Variantes didácticas	Escenarios disponibles	5
Cobertura del ejemplo	Nodos alcanzables visitados	7 de 7
No duplicación	Nodos procesados más de una vez	0
Dependencias	Paquetes externos en ejecución	0

Riesgos y respuesta

Riesgo	Prevención
Confundir BFS con DFS	Mostrar FIFO y niveles antes del código
Interpretar enlaces como distancia	Definir cercanía lógica como cantidad de conexiones
Ciclo infinito	Marcar visitados antes de encolar
Configuración inconsistente	Validar nodos, conexiones y neighbor_order
Mutación accidental	Copiar nodos y vecinos antes de cambiar un escenario
Falla en la exposición	Ejecutar pruebas y conservar salida de demostración

Límite técnico

BFS minimiza la cantidad de enlaces. No optimiza tiempo real, distancia, caudal, presión ni costo. Si los enlaces tuvieran pesos, correspondería evaluar otro algoritmo, como Dijkstra o costo uniforme.

8. Guion de exposición de 5 minutos

Tiempo	Expone	Mensaje
0:00-0:20	Santiago	Presentación del problema y objetivo
0:20-1:10	Santiago	Topología, cercanía lógica y niveles
1:10-2:05	Santiago	FIFO y pasos de BFS
2:05-3:20	Mateo	Implementación real y salida de la CLI
3:20-4:25	Mateo	Cinco escenarios, trece pruebas y límites
4:25-5:00	Mateo	Conclusión

Preguntas probables

¿Por qué BFS y no DFS? Porque completa los nodos cercanos antes de profundizar.

¿Qué significa ruta mínima? Menor cantidad de enlaces, no menor distancia física.

¿Qué evita repeticiones? El conjunto visited.

¿Qué sucede con un nodo aislado? Se informa en unreachable.

¿Es telemetría real? No, es una simulación académica.

Entregables actuales

Entregable	Contenido
Código	Paquete con red, BFS, escenarios y CLI
Datos	Topología externa data/red_sensores.json
Pruebas	Trece métodos unittest aprobados
Documentación	README, arquitectura, casos y guía paso a paso
Material de exposición	Este PDF y presentación PPTX de 5 minutos

Conclusión

La implementación cumple la búsqueda no informada solicitada: modela la conectividad, recorre por niveles y produce resultados verificables con código sencillo y documentado.

Referencias: documentación oficial de Python sobre collections.deque; Russell y Norvig, *Artificial Intelligence: A Modern Approach*; material de Uninformed Search de UC Berkeley CS188.